NATIONAL TECHNICAL UNIVERSITY OF UKRAINE
“IGOR SIKORSKY KYIV POLYTECHNIC INSTITUTE”
EDUCATIONAL AND RESEARCH INSTITUTE OF ATOMIC AND THERMAL ENERGY
DEPARTMENT OF DIGITAL TECHNOLOGIES IN ENERGY

CALCULATION AND GRAPHICS WORK

in the discipline “Methods of Virtual Reality Synthesis”

TOPIC: “Spatial Audio”

VARIANT: 5

Completed by:
student of group TR-51mp
Orest Bodnar

Reviewed by:
Anatoliy Demchyshyn

Kyiv – 2026

1. Task Description

This calculation and graphics work extends the functionality implemented in Practical Assignment 2 by adding spatial audio capabilities to the existing WebGL application. The application was previously developed to render a parametric surface with anaglyphic stereo projection and to control its orientation in real time using a smartphone as a tangible interface via the WebAudio rotation_vector sensor.

The objective of the present work is to implement spatial audio through the Web Audio HTML5 API. Unlike in PA#2, the three-dimensional surface remains stationary throughout the scene. Instead, a virtual sound source orbits around the geometrical centre of the surface patch, and its position in space is controlled by the smartphone's orientation. The user holds the phone and rotates it in any direction; the sound source moves accordingly, creating a perceivable shift in the stereo field when listening through headphones.

The specific requirements of the assignment are as follows. The application must reproduce a music track in MP3 format through a spatially positioned audio source whose three-dimensional coordinates are determined by the tangible interface. The position of the sound source must be visualised as a sphere rendered within the existing WebGL scene. Additionally, a BiquadFilterNode-based audio filter must be implemented and exposed to the user through a checkbox control that enables or disables the filter at runtime. According to variant 5, the required filter type is a high-shelf filter, which boosts or attenuates all frequencies above a specified shelf frequency. The report must also include a user manual with screenshots and a sample of the source code.

2. Theory

2.1 Spatial Audio and Binaural Perception

The human auditory system determines the direction of a sound source primarily through two mechanisms. The first is the interaural time difference (ITD): sound arriving from the left reaches the left ear slightly earlier than the right, and the brain interprets this temporal offset as a lateral direction cue. The second is the interaural level difference (ILD): the head and outer ear partially attenuate the signal reaching the far ear, so a sound from the left is perceived as louder in the left ear. Together, ITD and ILD provide accurate left-right and front-back localisation, particularly at different frequency ranges.

A more complete model of spatial perception is described by the head-related transfer function (HRTF). The HRTF is a pair of frequency-domain filters that capture how sound is modified by the shape of the head, pinnae, and torso before reaching each ear canal. Convolving a mono audio signal with the appropriate HRTF pair for a given source direction produces a binaural signal that, when played back through headphones, creates a convincing three-dimensional auditory impression. The Web Audio API implements this model natively through the `PannerNode` when its `panningModel` property is set to `'HRTF'`.

2.2 Web Audio API

The Web Audio API models audio processing as a directed graph of nodes. Each node performs a specific operation on the audio signal, and nodes are connected via their input and output ports to form a processing chain. All nodes belong to an `AudioContext`, which manages the audio hardware and timing.

The `AudioBufferSourceNode` loads a decoded PCM buffer and streams it to its output. Setting its `loop` property to `true` causes the buffer to repeat indefinitely. Once stopped, a source node cannot be restarted; a new instance

must be created for each playback session. Pausing is implemented by suspending the `AudioContext` itself, which freezes the entire graph at its current state and resumes from the same position on `context.resume()`.

The `BiquadFilterNode` implements a variety of second-order IIR filters selectable by its `type` property. The high-shelf filter (`type = 'highshelf'`) applies a constant gain boost or cut to all frequencies above the shelf frequency, with a smooth transition in the band around the cutoff. The amount of boost or attenuation is controlled by the `gain.value` property in decibels. Setting `gain.value` to zero makes the filter transparent, which provides a clean bypass without disconnecting the node from the graph.

The `PannerNode` spatialises a mono audio stream relative to the `AudioListener` position. When `panningModel` is `'HRTF'`, the node applies a measured HRTF dataset to produce the binaural signal described above. The source position is updated by calling `setPosition(x, y, z)` on the node. The `ChannelSplitterNode` separates the stereo output into individual mono channels, making it possible to attach independent `AnalyserNode` instances to the left and right streams for visualisation.

2.3 Rotation Vector Sensor and Quaternion-to-Matrix Conversion

The Android `rotation_vector` sensor is a software-fused sensor that combines readings from the accelerometer, gyroscope, and magnetometer to produce a stable, drift-free estimate of the device orientation. The output is a unit quaternion expressed as the vector $[x,\, y,\, z,\, w]$. Because the sensor fuses all three physical sensors, it is more robust than any single hardware sensor and provides reliable orientation data suitable for use as a real-time control signal.

A quaternion $(q_0,\, q_1,\, q_2,\, q_3)$, where $q_0 = w$, is converted to a 4×4 column-major rotation matrix suitable for WebGL using the standard expansion of the quaternion product. The matrix elements are formed from sums and differences of

products of the quaternion components, and the conversion is ported directly from Android's `SensorManager.getRotationMatrixFromVector` implementation. The resulting matrix R rotates any vector from the sensor's initial reference frame to the current device orientation. Applying R to a fixed base point, such as $[2,\ 0,\ 0,\ 1]$, produces the current position of the virtual sound source on a sphere of radius 2 centred at the origin.

3. Implementation

3.1 Audio Processing Chain

All audio functionality is encapsulated in the `AudioManager` class. The class creates a single `AudioContext` on construction and builds the processing graph when the audio file is loaded. The chain is: `AudioBufferSourceNode` $\rightarrow$ `BiquadFilterNode` $\rightarrow$ `PannerNode` $\rightarrow$ `AudioContext.destination`. In addition, the panner output is connected to a `ChannelSplitterNode` whose two outputs feed individual `AnalyserNode` instances for the left and right channels.

Playback is managed through three public methods. `play()` resumes a suspended context if the audio was previously paused, or creates a new `AudioBufferSourceNode` and starts it from the beginning otherwise. `pause()` suspends the `AudioContext`, freezing the playback position. `restart()` stops any running source, resumes the context, and creates a fresh source node starting from offset zero. This design ensures that a single `AudioContext` persists for the entire lifetime of the page, avoiding the browser restriction that limits the number of simultaneously active contexts.

3.2 Sound Source Positioning

The `SensorConnection` class, inherited from PA#2, connects to the Sensor Server application on the smartphone over a WebSocket and delivers

rotation matrices via a callback. In the CGW version of the application, the callback no longer applies the matrix to the surface view transform. Instead, it transforms the fixed base point $[2, 0, 0, 1]$ by the received rotation matrix using `m4.transformVector`, and stores the resulting three-dimensional coordinates as `renderer.sphereTarget`.

To avoid abrupt jumps caused by sensor noise or irregular delivery intervals, the sphere position is smoothed using an exponential moving average applied in the render loop rather than in the sensor callback. Each frame, the current position `spherePos` is advanced toward `sphereTarget` by a factor of $\alpha = 0.15$: `pos` $+= \alpha \cdot$ (`target` $-$ `pos`). Applying the smoothing at 60 fps rather than at the sensor rate of approximately 20 Hz provides rapid convergence while eliminating visible discontinuities. The smoothed position is then passed to `AudioManager.setPannerPosition(x, y, z)` on every frame, keeping the audio source synchronised with the visual sphere.

3.3 Stereo Sphere Rendering and Level Metering

The sound source sphere is rendered using the existing `SphereModel` class. A `drawStereo` method was added to the class that binds only the vertex buffer, since the stereo shader does not use normals or texture coordinates. In the `renderStereo` method of the `Renderer`, after drawing the surface for each eye, the model-view matrix is recomputed to include the sphere's current world-space translation applied after the base view transform. This places the sphere at the correct position in the scene for both the red and cyan eye passes, producing an anaglyphic representation consistent with the rest of the scene.

Left and right channel waveforms are drawn on a `canvas` element inside the control panel each frame. The `Application.drawMeters` method reads time-domain data from each `AnalyserNode` using `getByteTimeDomainData`, normalises the samples to the range $[-1, 1]$, applies a visual gain factor of 3, and

renders the waveform as a smooth curve using quadratic Bézier segments. When the sound source is positioned to one side of the listener, the corresponding channel carries more energy and its waveform visibly exhibits greater amplitude than the opposite channel.

4. User Manual

4.1 Opening the Application

The application runs entirely in the browser and requires no installation or build step. Open the file `index.html` from the project root using a local HTTP server (for example, `python -m http.server`) or a browser extension such as Live Server. A static file server is required because the application loads shader files and the audio track via `fetch`, which browsers block on `file://` URLs. Upon loading, the WebGL canvas initialises with the parametric surface and anaglyphic stereo rendering active. The interface is shown in Figure 4.1.

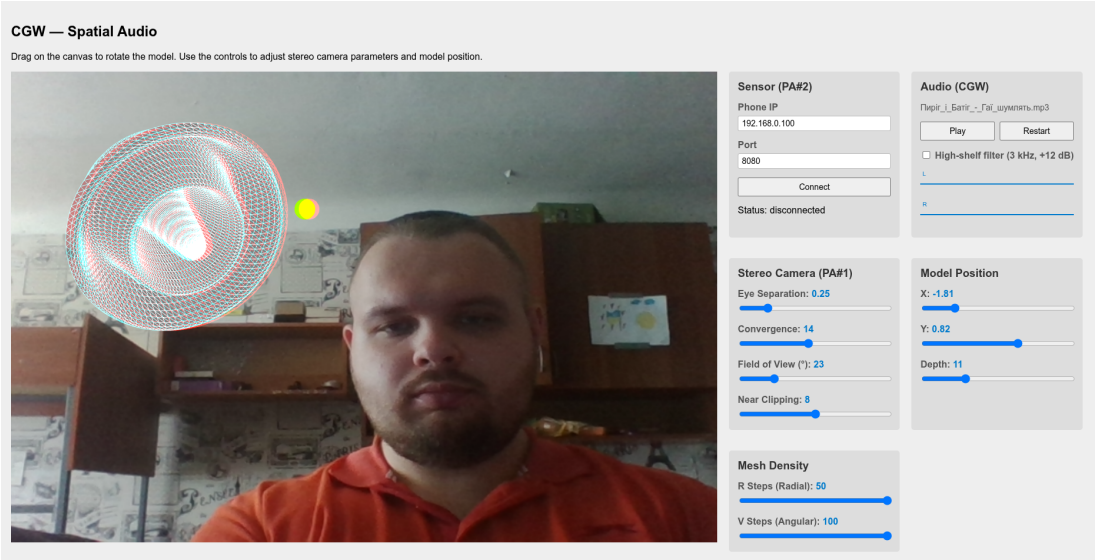

Fig. 4.1 — Application on startup

4.2 Connecting the Smartphone

Install the Sensor Server application on an Android device and start the server. Ensure the phone and the computer are on the same Wi-Fi network. In the *Sensor* control group, enter the phone's local IP address and port (default 8080), then press *Connect*. The status label changes to *connected* and the yellow sphere begins to move in response to the phone's orientation. Rotating the phone in any direction causes the sphere to orbit around the surface centre accordingly. Figure 4.2 shows the application with the sensor connected and the sphere displaced from its default position.

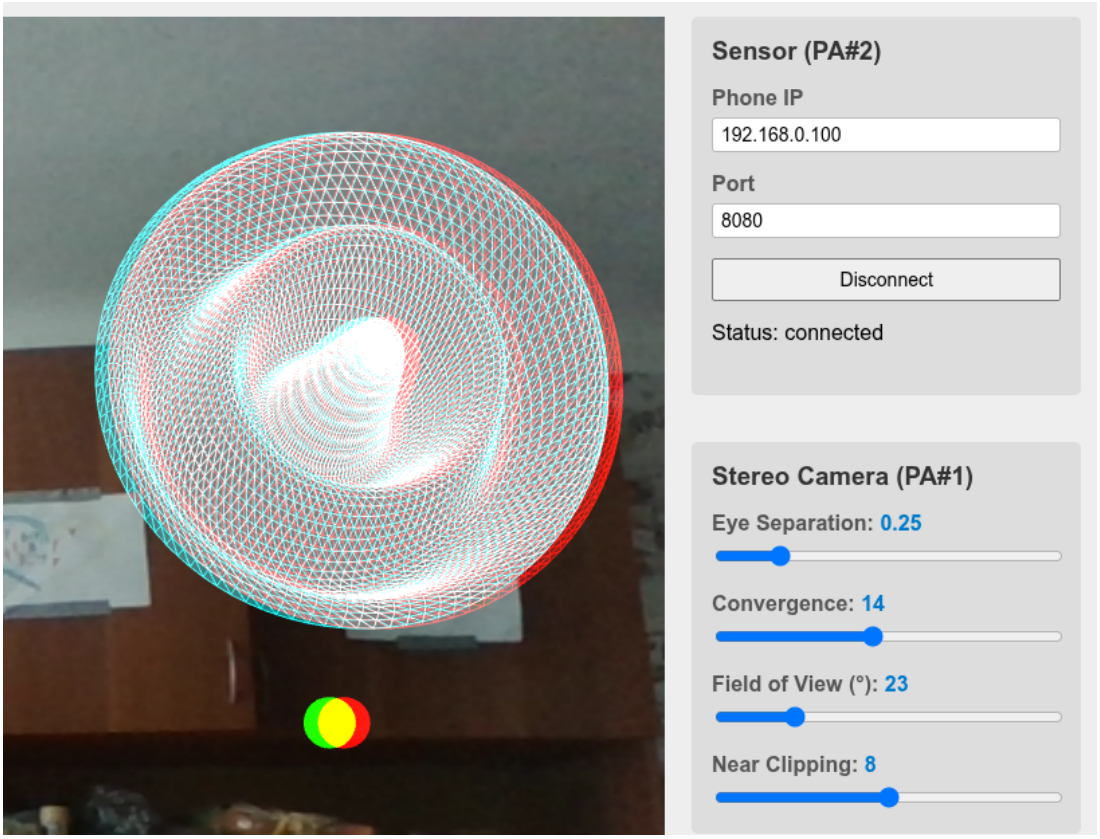

Fig. 4.2 — Sensor connected and sphere orbiting the surface

4.3 Playing Audio and Observing the Stereo Field

Press the *Play* button in the *Audio* control group to start playback. The button label changes to *Pause*, and the two waveform charts below become active, showing

the time-domain signal for the left and right channels separately. Put on headphones and rotate the phone: as the sphere moves to the left, the sound is perceived from the left and the left waveform shows greater amplitude; moving the sphere to the right produces the opposite effect. The *Pause* button suspends playback without losing the current position; pressing it again resumes from the same point. The *Restart* button returns to the beginning of the track. Figure 4.3 illustrates the waveforms with the sphere positioned off-centre.

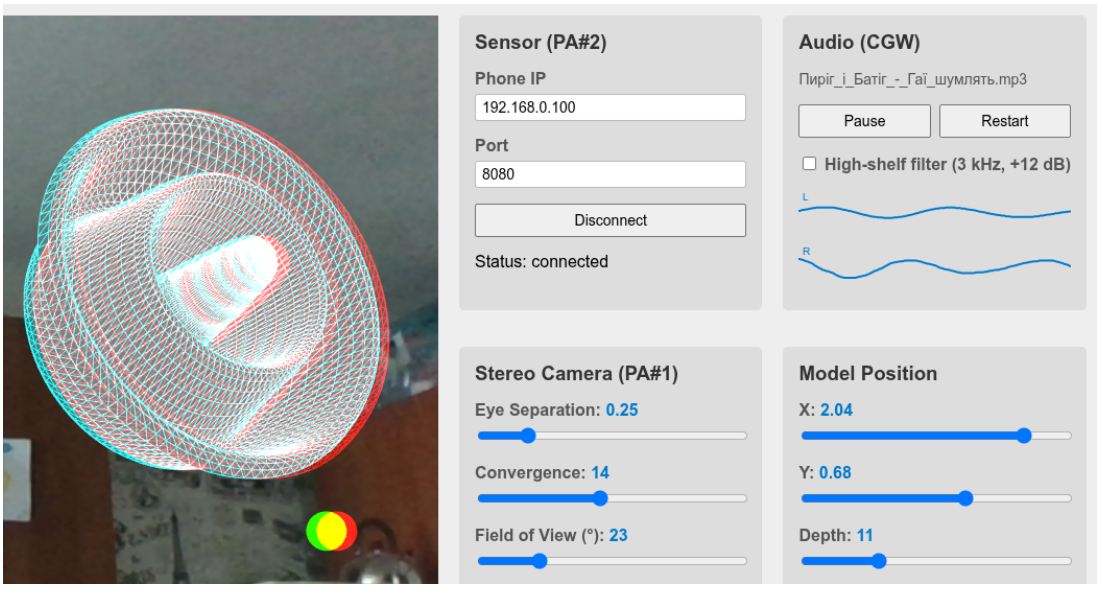

Fig. 4.3 — Audio playing with left and right channel waveforms

4.4 Using the High-Shelf Filter

Tick the *High-shelf filter (3 kHz, +12 dB)* checkbox to enable the filter. The high-shelf filter boosts all frequencies above 3000 Hz by 12 dB, resulting in a noticeably brighter, more present sound character. Unticking the checkbox sets the filter gain to zero, making it transparent without removing it from the signal chain. The difference is most apparent on content with prominent high-frequency content such as cymbals or vocals. Figure 4.4 shows the interface with the filter enabled.

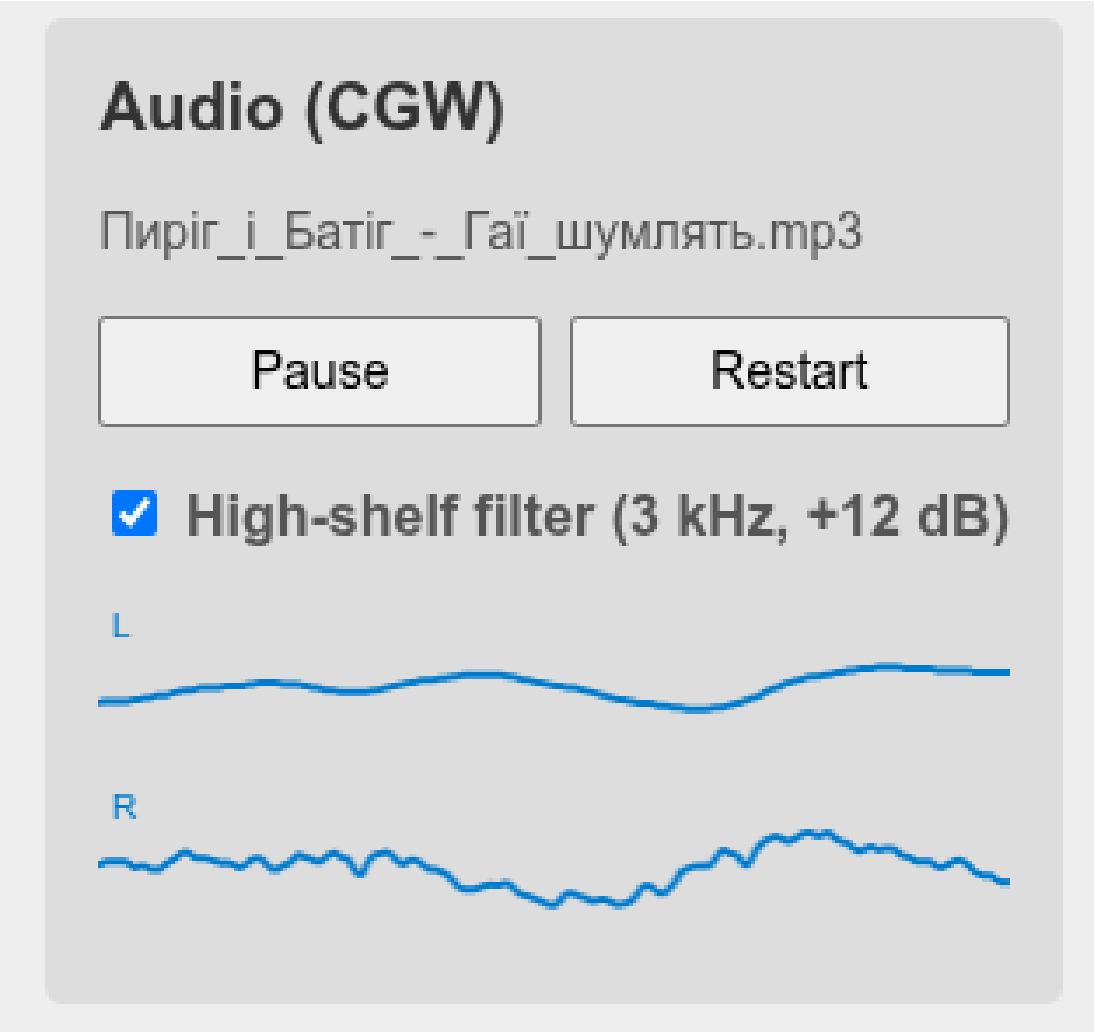

Fig. 4.4 — High-shelf filter enabled

5. Source Code

Appendix A. AudioManager.js

```
class AudioManager {
    constructor() {
        this.context = new AudioContext();
        this.buffer = null;
        this.source = null;
        this.panner = null;
        this.isPlaying = false;
        this.isPaused = false;
    }
```

```
async load(url) {
    const response = await fetch(url);
    const arrayBuffer = await response.arrayBuffer();
    this.buffer = await this.context.decodeAudioData(arrayBuffer);

    this.filter = this.context.createBiquadFilter();
    this.filter.type = 'highshelf';
    this.filter.frequency.value = 3000;
    this.filter.gain.value = 0;

    this.panner = this.context.createPanner();
    this.panner.panningModel = 'HRTF';
    this.panner.distanceModel = 'inverse';
    this.panner.refDistance = 1;

    this.filter.connect(this.panner);
    this.panner.connect(this.context.destination);

    const splitter = this.context.createChannelSplitter(2);
    this.panner.connect(splitter);

    this.analyserL = this.context.createAnalyser();
    this.analyserR = this.context.createAnalyser();
    this.analyserL.fftSize = 256;
    this.analyserR.fftSize = 256;
    splitter.connect(this.analyserL, 0);
    splitter.connect(this.analyserR, 1);
}

async play() {
    if (!this.buffer) return;
    if (this.isPaused) {
        await this.context.resume();
        this.isPaused = false;
        this.isPlaying = true;
    } else if (!this.isPlaying) {
        await this._startSource();
    }
```

```
}

async pause() {
    if (!this.isPlaying) return;
    await this.context.suspend();
    this.isPlaying = false;
    this.isPaused = true;
}

async restart() {
    if (this.source) {
        this.source.stop();
        this.source = null;
    }
    await this.context.resume();
    this.isPaused = false;
    await this._startSource();
}

setPannerPosition(x, y, z) {
    if (this.panner) this.panner.setPosition(x, y, z);
}

setFilterEnabled(enabled) {
    if (this.filter) this.filter.gain.value = enabled ? 12 : 0;
}

getLevels() {
    if (!this.analyserL || !this.analyserR) return { left: 0, right: 0 };
    return { left: this._rms(this.analyserL), right: this._rms(this.
analyserR) };
}

_rms(analyser) {
    const data = new Uint8Array(analyser.fftSize);
    analyser.getByteTimeDomainData(data);
    let sum = 0;
    for (let i = 0; i < data.length; i++) {
        const v = (data[i] - 128) / 128;
```

```
            sum += v * v;
        }
        return Math.sqrt(sum / data.length);
    }

    async _startSource() {
        await this.context.resume();
        this.source = this.context.createBufferSource();
        this.source.buffer = this.buffer;
        this.source.loop = true;
        this.source.connect(this.filter);
        this.source.start(0);
        this.isPlaying = true;
    }
}
```